AJAY KADIYALA - Data Engineer

Follow me Here:

LinkedIn:

https://www.linkedin.com/in/ajay026/

Data Geeks Community:

https://lnkd.in/geknpM4i

Data Engineer Behavioral + Leadership Interview Playbook

DataGeeks

COPY RIGHTS RESERVED

Table of Contents

1. **How you will use this playbook**
2. **What interviewers actually grade in behavioral rounds**
3. **Your story frameworks**
4. **Your personal Story Bank**
5. **Leadership questions (Data Engineer specific)**
6. **Stakeholder + business communication (where most people fail)**
7. **Stakeholder + business communication (where most people fail)**
8. **Failure + mistake questions (how you answer safely)**
9. **Ownership + ambiguity questions**
10. **Collaboration questions (cross-team + team culture)**
11. **Behavioral question bank**
12. **10 Real Scenarios with detailed "your answers"**
13. **"Ask the interviewer" questions**
14. **Your 30-minute prep before any interview**
15. **Cheat sheets (1-page summaries)**

0.1 Your goal (what you are actually trying to win)

In behavioral rounds, you are not proving you are smart.
You are proving you are:

- safe to trust in production
- clear in communication
- strong in ownership
- calm under pressure
- mature with stakeholders

So your answers must sound like someone who has **already done the work**.

0.2 Your rule: you will speak in "story + impact" (not theory)

Whenever you get a question, you will answer like this:

DataGeeks

COPY RIGHTS RESERVED

One-liner: what happened + what you did
3 bullets: what actions you took
Metric: what improved (time, cost, quality, SLA)

Example structure you will use:

- "I handled X by doing Y, and it resulted in Z."
- "I took three steps…"
- "The outcome was…"

This stops you from rambling.

0.3 How you will practice (10–15 minutes daily plan)

Day plan (simple)

Step 1 (2 mins): pick 1 question from Section 10
Step 2 (5 mins): write your answer in STAR/CARE format
Step 3 (3 mins): speak it out loud once
Step 4 (3 mins): tighten it to 60–90 seconds
Step 5 (2 mins): add 1 metric or proof point

You will do this daily for 14 days and your answers will become automatic.

0.4 Your "Story Bank" approach (you will build once, reuse everywhere)

You will not create 80 unique answers.
You will build **12 strong stories**, and reuse them across questions.

Because the same story can answer:

- ownership
- conflict
- failure
- leadership
- stakeholder management
  …just from a different angle.

DataGeeks

COPY RIGHTS RESERVED

0.5 Your Story Bank template (copy and fill)

For each story, you will fill this:

- **Title:** (example: “Revenue dashboard spike incident”)
- **Context:** what system + what problem
- **Your role:** what you owned
- **Trigger:** what went wrong / what was asked
- **Actions:** 3–5 concrete steps you took
- **Result:** metric + business impact
- **Lesson:** what you changed to prevent recurrence
- **Proof:** link to dashboard/log/Jira/PR (if you can mention it)

This makes your stories consistent and believable.

0.6 Your speaking rules (so you sound confident)

You will follow these rules:

- Keep answer **60–90 seconds** unless they ask follow-ups
- Speak in **simple English**
- Use **“I” statements** (I did, I drove, I aligned)
- Use **numbers** whenever possible (%, mins, hrs, cost)
- Avoid deep tech jargon unless they ask

0.7 What you will do when you don’t know an answer

You will never say “I don’t know” and stop.

You will say:

- “I haven’t faced that exact case, but I will explain how I would approach it.”
  Then you give:

DataGeeks

COPY RIGHTS RESERVED

1. how you clarify requirements
2. how you reduce risk (POC, rollback, monitoring)
3. how you deliver incrementally

This still scores well.

0.8 The 3 "bad patterns" you will remove

Remove these from your answers:

1. **Blaming others**

- "They gave wrong requirement…"
  Instead say:
- "I clarified and aligned expectations."

2. **Vague statements**

- "I improved performance a lot…"
  Instead say:
- "I reduced runtime from 8 hours to 90 minutes."

3. **Too much story, no result**

- You must end with outcome + lesson.

0.9 Your quick warm-up drill (before any interview)

You will do this 10-minute drill:

- Pick 3 stories from your bank
- Speak each story in 45 seconds
- Add 1 metric for each
- Add 1 lesson for each

You will enter the interview with confidence.

0.10 Mini exercise (you will do now)

DataGeeks

COPY RIGHTS RESERVED

Pick 1 story you already have from your work life (any one):

- incident
- migration
- cost save
- data quality bug
- stakeholder conflict

Tell me the story title in one line.

1) "In behavioral rounds, what are they *really* grading you on?"

Your ready answer (say this):
They're grading **impact + ownership + clarity + maturity**. You don't need a perfect story. You need a clear problem, what *you* did, and what changed because of it—ideally with a number (time, cost, quality, reliability).

2) "What does a 'hidden scoring rubric' look like in simple words?"

Your ready answer:
Think of 4 checks:

1. **Impact**: did the work matter?
2. **Ownership**: did you drive it end-to-end?
3. **Clarity**: can you explain without rambling?
4. **Maturity**: did you handle tradeoffs + people well?

3) "Why do technical-only answers fail in behavioral rounds?"

Your ready answer:
Because tech alone doesn't show **decision-making**. If you only say "I used Spark/ADF/dbt," it sounds like a tool demo. You must add: **why this approach**, **what risk you managed**, and **how you aligned stakeholders**.

DataGeeks

COPY RIGHTS RESERVED

4) “How do you prove impact as a Data Engineer (not vague words)?”

Your ready answer:
Use one metric. Example format:
“One-liner: I improved pipeline reliability.”
Then: “Reduced failures from **X/week → Y/week**, improved SLA from **T+1 to T+0**, and added data checks that cut bad records by **N%**.”
(Replace X/Y/T/N with your real numbers.)

5) “How do you show ownership without sounding arrogant?”

Your ready answer:
Say “I owned” the **process**, not the people:
“I aligned requirements, designed the approach, shipped in small releases, monitored production, and wrote runbooks. I also pulled in the right stakeholders early to avoid rework.”

6) “What does good clarity sound like (so you don’t ramble)?”

Your ready answer:
Use **60–90 seconds**:

- 1 line: situation + goal
- 3 bullets: your actions (in order)
- 1 line: result (metric + business effect)
  If you feel you’re talking too long, stop after the metric.

7) “What does ‘senior mindset’ sound like even if you’re mid/junior?”

Your ready answer:
Senior mindset is: **think ahead + reduce risk + make others successful**.
“I didn’t just build the pipeline. I added monitoring, alerts, rollback plan, data quality checks, and documentation so the system survives without me.”

8) “How do you show maturity when things were unclear or messy?”

DataGeeks

COPY RIGHTS RESERVED

Your ready answer:
You show **structured thinking**:
"I confirmed the goal, wrote assumptions, proposed 2 options with tradeoffs, got quick sign-off, and delivered in phases. When scope changed, I re-baselined timeline with reasons."

9) "How do you explain a decision/tradeoff (speed vs quality vs cost)?"

Your ready answer:
"I made the tradeoff explicit: If we ship fast, risk is X; if we harden quality, time is Y. I chose a middle path—ship MVP with guardrails (DQ checks, retries, alerts), then optimize cost/performance in the next sprint."

10) "What do you say if you don't know an answer in the interview?"

Your ready answer:
"I haven't done that exact thing, but here's how I'd approach it: clarify requirements, check constraints, propose options, validate with a small test, and monitor after release. If needed, I'll partner with the right SME and close the gap fast."

1) "What is STAR, and why does it work in behavioral rounds?"

Your ready answer (say this):
STAR keeps you structured: **Situation** (context), **Task** (goal), **Action** (what you did), **Result** (what changed). Interviewers like it because it shows *logic* + *ownership* + *outcome* without rambling.

2) "How do you keep STAR short (so you don't over-explain)?**

Your ready answer:
I keep **S+T to 15–20 seconds**, **A to 30–40 seconds**, **R to 10–15 seconds**. If I add details, I only add them when asked in follow-ups.

3) "What is CARE, and when should you use it instead of STAR?"

DataGeeks

COPY RIGHTS RESERVED

Your ready answer:
CARE is faster: **Context → Action → Result → Evidence**. I use CARE when the situation is complex but I want to highlight what I did and prove it with evidence (metrics, dashboards, alerts, cost reports, postmortem).

4) "How do you choose between STAR vs CARE in real interviews?"

Your ready answer:
If the question is about **behavior/conflict/decision**, I use **STAR** (clear story flow).
If the question is about **impact/ownership/optimization**, I use **CARE** (results + proof quickly).

5) "What is the 'One-liner → 3 bullets → metric' structure?"

Your ready answer:
It's my default speaking style:

- **One-liner:** what you delivered / solved
- **3 bullets:** the key actions you took (in order)
- **Metric:** one number + business effect
  It sounds confident and avoids long stories.

6) "What do you use as 'Evidence' so it doesn't look like you're bluffing?"

Your ready answer:
Evidence can be: pipeline run history, SLA dashboard, incident ticket, cost report, DQ checks, alert metrics, query runtime stats, before/after charts, or stakeholder sign-off. I mention just **one** proof point, not a list.

7) "What exactly is the 60–90 second rule and how do you follow it?"

Your ready answer:
I aim to finish my first answer in **under 90 seconds**. I stop right after the result. If the interviewer wants depth, they'll ask. That's how you sound senior—clear first, details later.

DataGeeks

COPY RIGHTS RESERVED

8) “How do you make your ‘Action’ part strong (so it doesn’t sound generic)?”

Your ready answer:
I speak in **decision + steps**:
“I chose X because Y, then I did 1,2,3.”
I avoid “we did” and say what *I personally owned* (design, validation, rollout, monitoring, comms).

9) “How do you add metrics if you don’t have exact numbers?”

Your ready answer:
I use a **safe range** or **proxy**:
“Cut runtime roughly by ~30–40%” or “reduced failures from frequent to rare (weekly → monthly).”
Then I anchor it with evidence like pipeline logs or incident counts.

10) “How do you avoid jargon and still sound technical enough?”

Your ready answer:
I explain tools as outcomes:
Instead of “I used Spark + Delta,” I say “I processed large data reliably, added checks, and improved runtime/cost.” Tools come after, only if asked or if it strengthens the proof.

1) “What is a ‘story bank’ and why do you need it?”

Your ready answer (say this):
A story bank is a **small set of real work stories** you can reuse for 80% of behavioral questions. Instead of inventing answers in the interview, you pick the closest story, adjust 1–2 lines, and deliver confidently.

2) “What are the 12 must-have stories for a Data Engineer?”

Your ready answer:
You prepare:

- **3 wins** (impact, scale, cost/time saved)
- **2 failures** (lesson + fix)

DataGeeks

COPY RIGHTS RESERVED

- **2 conflicts** (stakeholder / teammate / manager)
- **2 ambiguity** (unclear requirements, scope change)
- **1 leadership** (mentoring / driving standards / ownership)
- **1 incident** (on-call, outage, prevention)
- **1 migration/optimization** (legacy → modern)

3) "How do you pick your '3 wins' so they sound strong?"

Your ready answer:
Pick wins that show **business impact + engineering depth**:

1. reliability (SLA, fewer failures)
2. performance (runtime reduced)
3. cost (cloud spend reduced)
   Each win needs **one metric** and **one stakeholder outcome**.

4) "How do you talk about failures without hurting your chances?"

Your ready answer:
You keep it safe:
"I made a mistake, owned it fast, fixed the impact, and added prevention."
Avoid blaming. Show: **root cause + guardrails (tests/DQ/alerts/runbook)**.

5) "What's the best way to write each story in 6 lines (so it's reusable)?"

Your ready answer:
Use this mini-template (you fill blanks):

- Context: ___
- Goal: ___
- What you did (3 bullets): 1) ___ 2) ___ 3) ___
- Result metric: ___
- Evidence: ___ (dashboard/log/incident/cost report)

DataGeeks

COPY RIGHTS RESERVED

6) “How do you make each story sound like *you* owned it?”

Your ready answer:
Mention the parts you owned end-to-end:
“I clarified requirements, proposed options, implemented, validated, rolled out, monitored, and documented.”
Even if a team worked on it, you highlight **your slice** clearly.

7) “What metrics are best for Data Engineering stories?”

Your ready answer:
Use any one of these:

- SLA/freshness (T+1 → T+0)
- runtime (8h → 90m)
- failure rate (daily failures → rare)
- cost (monthly spend down by X%)
- data quality (bad records down by X%)
- adoption (users/teams onboarded)

8) “How do you handle confidentiality while still sounding credible?”

Your ready answer:
You generalize names and data:
“I worked for a telecom client” / “large retail platform.”
Use ranges instead of exact numbers if needed. Keep the **problem + approach + outcome** real.

9) “How do you map one story to multiple questions?”

Your ready answer:
You change the **opening line** based on the question:

- If they ask leadership → emphasize mentoring/standards.

DataGeeks

COPY RIGHTS RESERVED

- If they ask conflict → emphasize alignment + pushback.
- If they ask ownership → emphasize end-to-end delivery.
  Same story, different angle.

10) "How do you practice so your story comes out in 60–90 seconds?"

Your ready answer:
You rehearse using:
One-liner → 3 bullets → metric.
Record yourself once. Remove extra details. Stop after the metric. Save deeper details for follow-ups.

1) "Tell me about a time you showed ownership without authority."

Your ready answer:
I saw a recurring pipeline failure nobody owned clearly. I set up a quick huddle with analyst + upstream app team, defined one owner per step, and created a small fix plan. I shipped the first stabilizing patch, added alerts + runbook, and then handed clear ownership with SLAs. Failures dropped and the team stopped firefighting.

2) "How do you mentor a junior Data Engineer?"

Your ready answer:
I don't just correct code—I teach thinking. I pair once, explain the 'why', and give them a small piece to own. I review PRs with clear comments: what's risky, what's good, what to improve next time. Over 2–3 sprints, they start shipping cleaner pipelines with less rework.

3) "How do you do code reviews for data pipelines (SQL/PySpark/dbt)?"

Your ready answer:
I review in 4 layers: correctness, performance, reliability, and readability. I check joins, duplicates, null handling, incremental logic, and edge cases. Then I check partitioning/clustering, pushdown, caching, and cost hotspots. Finally, I ensure tests, logging, and clear naming so someone else can maintain it.

DataGeeks

COPY RIGHTS RESERVED

4) “How do you drive standards like Data Quality and CI/CD?”

Your ready answer:
I start small: define a few mandatory checks (nulls, uniqueness, freshness) and make them part of the pipeline. Then I standardize templates—branching, PR checks, deployment steps, rollback plan. I make it easy to follow and hard to bypass. That’s how standards stick without forcing people.

5) “How do you handle delivery pressure and tight timelines?”

Your ready answer:
I split work into MVP + hardening. First, I deliver the minimum that solves the business need with guardrails (basic DQ checks, retries, alerts). Then I plan the next sprint for performance tuning, extra tests, and documentation. I communicate tradeoffs early so there are no surprises.

6) “Tell me about a time you made a tough tradeoff (speed vs quality vs cost).”

Your ready answer:
We had a deadline, but the full solution would take longer. I proposed two options: fast delivery with controlled risk, or slower delivery with full coverage. We chose fast delivery but I added checkpoints—data validation, monitoring, and rollback. We shipped on time and avoided production issues.

7) “How do you influence stakeholders when they want ‘everything now’?”

Your ready answer:
I don’t say no directly. I ask what matters most—accuracy, speed, or cost. Then I show impact: “If we add this now, timeline moves by X days.” I offer a phased plan: deliver top priority first, then iterate. People accept pushback when you give a clear path forward.

8) “How do you ensure long-term maintainability, not just ‘it works’?”

Your ready answer:
I design for future changes: modular pipelines, clear contracts, metadata-driven configs where possible, and consistent naming. I add tests, monitoring, and documentation as

DataGeeks

COPY RIGHTS RESERVED

part of done. My goal is: if I'm not available, the pipeline still runs and others can debug it fast.

9) "How do you lead during an incident or production issue?"

Your ready answer:
First I stabilize: stop the blast radius, rollback or hotfix, and communicate status. Then I do root cause properly, not guesswork. After recovery, I add prevention: alerts, retries, data checks, and a short postmortem with action items. The key is calm communication + permanent fixes.

10) "How do you measure your leadership impact as a Data Engineer?"

Your ready answer:
I look at outcomes: fewer incidents, faster delivery with less rework, cleaner PR cycles, better on-call health, and clearer stakeholder trust. If the team can ship reliably without constant firefighting—and juniors grow faster—that's real leadership impact.

1) "How do you convert a business requirement into a data requirement?"

Your ready answer (say this):
First I restate the business goal in plain words. Then I convert it into **three data specs**:

1. **Metric definition** (what exactly is "revenue/active user/conversion")
2. **Grain** (per customer? per order? per day?)
3. **Freshness + accuracy** (SLA, tolerances, exclusions)
   Then I validate with a quick example row or sample dashboard.

2) "How do you handle vague requirements from PM/Analyst?"

Your ready answer:
I don't start building immediately. I ask for **one example**: "Can you show one record or one report you expect?"
Then I write assumptions: "If we define X as Y, is that correct?"
I get a quick yes in writing, and only then I implement. This reduces rework and protects timeline.

DataGeeks

COPY RIGHTS RESERVED

3) “How do you handle scope creep politely?”

Your ready answer:
I acknowledge the request, then I anchor on impact:
“Totally doable. If we add this now, it adds ~X days or we drop Y from scope. Which is more important for this release?”
If needed, I create a Phase 2 list and commit to a date, so it doesn’t feel like rejection.

4) “How do you explain delays without sounding weak?”

Your ready answer:
I explain **facts + plan + new date**, not emotions.
“Root cause is upstream schema change + data mismatch. I’ve already applied a patch and added a validation check. New ETA is tomorrow 3 PM. I’ll send an update at 1 PM.”
This sounds controlled, not defensive.

5) “How do you manage expectations on SLA, freshness, and data quality?”

Your ready answer:
I define SLAs in simple terms:

- Freshness: “Data available by **T+0 / 9 AM daily**”
- Quality: “Nulls < X%, duplicates = 0 for keys”
- Failure handling: “If SLA breaks, alert triggers + fallback view”
  Stakeholders trust you when expectations are explicit.

6) “How do you align with PM/Analyst/DS/Engineering without confusion?”

Your ready answer:
I set **clear ownership**:

- Analyst: metric definition + acceptance
- Engineering/App team: source contract + schema change notice

DataGeeks

COPY RIGHTS RESERVED

- Me: pipeline + model + tests + monitoring
  I also keep one shared doc (definitions + SLA + lineage). This avoids blame games later.

7) "How do you communicate technical issues to non-technical stakeholders?"

Your ready answer:
I avoid tool names first. I speak in outcomes:
"The data is delayed because the input format changed unexpectedly. We've fixed the processing and added a guardrail so this doesn't repeat."
If they ask deeper, then I mention the tool detail (ADF/Databricks/dbt/Snowflake).

8) "How do you push back when a stakeholder asks for an impossible timeline?"

Your ready answer:
I don't say "not possible." I offer options:
"Option A: deliver core metrics by Friday with basic checks. Option B: full scope by next Wednesday with stronger validation and backfill."
Then I ask them to pick. Pushback becomes a decision, not a conflict.

9) "How do you handle last-minute changes right before release?"

Your ready answer:
I follow a release rule:
"If change impacts schema/logic, we either (1) ship behind a flag, or (2) move it to next release."
I explain risk clearly: "Shipping now may break downstream dashboards."
Stakeholders usually accept when you frame risk to business.

10) "Tell me about a time you had to simplify complex data logic for business users."

Your ready answer:
I created a short definitions page: what each metric means, what's excluded, and when it updates. Then I walked them through one example and how the number is formed. After that, I got fewer ad-hoc questions and fewer 'number mismatch' escalations because everyone aligned on the same definition.

DataGeeks

COPY RIGHTS RESERVED

1) “Tell me about a time you had conflict with a teammate.”

Your ready answer (say this):
We disagreed on how to implement a pipeline change. I asked for 15 minutes, aligned on the goal, and listed 2 options with risks. We picked the safer approach for production, and I documented the decision in the PR. Result: no rollback needed, and the teamwork stayed positive.

2) “How do you handle conflict with an analyst/PM when requirements keep changing?”

Your ready answer:
I stay polite but firm. I say: “Happy to add this, but it changes scope. What should we de-prioritize?”
Then I write the updated requirement in one line and get a quick confirmation. This stops endless rework and keeps trust.

3) “What do you do when you disagree on design (you think it will break later)?”

Your ready answer:
I don’t argue emotionally. I present **risk + evidence**:
“If we do it this way, we may get duplicates / SLA misses / cost spike.”
Then I offer an alternative and a small test plan. If the team still chooses the other path, I align and document the tradeoff.

4) “How do you push back on an unrealistic deadline without sounding rude?”

Your ready answer:
I use options, not a flat ‘no’:
“Option A: deliver MVP by Friday with basic checks. Option B: full hardening by next Wednesday.”
Then I ask them to choose based on business priority. It becomes a decision, not a fight.

5) “Conflict with your manager: you think the priority is wrong. What do you do?”

Your ready answer:
I clarify impact first: “If we prioritize X, Y will slip and it affects SLA.”

DataGeeks

COPY RIGHTS RESERVED

I propose a simple re-order with reasons and ask for confirmation. If they still decide otherwise, I commit fully and ask what ‘done’ looks like to avoid confusion.

6) “When someone blames your pipeline for bad numbers, how do you handle it?”

Your ready answer:
I stay calm and go factual:
“Let’s validate source vs transformed vs dashboard.”
I check lineage, freshness, and DQ checks. If it’s my issue, I own it and fix fast. If it’s upstream or definition mismatch, I show proof and align everyone on the correct definition.

7) “How do you handle a teammate who keeps blocking your PRs or nitpicking?”

Your ready answer:
I ask for a quick sync and align on review standards: what’s must-have vs nice-to-have. Then I propose a checklist (naming, tests, performance, DQ). This reduces random comments and speeds up approvals without ego clashes.

8) “How do you handle conflict across teams (upstream app team won’t fix their data)?**

Your ready answer:
I escalate the right way: I share examples, impact, and a clear ask.
“If this field is null, downstream billing report breaks.”
I also add a temporary guardrail on my side (validation + fallback) while pushing for a permanent fix with owners and dates.

9) “How do you handle conflict when someone is being aggressive in meetings?”

Your ready answer:
I don’t match the tone. I reset the conversation:
“I want to solve this together. Let’s stick to facts and options.”
If needed, I move it offline: “Let’s take 10 mins after the call.” This keeps professionalism and avoids public fights.

DataGeeks

COPY RIGHTS RESERVED

10) “What is your default ‘conflict template’ you follow every time?”

Your ready answer:

1. Align on goal
2. Share facts/evidence
3. Offer 2 options with tradeoffs
4. Agree on decision + next steps
5. Document it (ticket/PR/notes)
   This makes conflict productive and prevents repeat issues.

1) “Tell me about a failure.”

Your ready answer (say this):
I once shipped a pipeline change that looked correct in dev but caused a downstream mismatch in production. I owned it immediately, paused the release, and aligned with the analyst on the expected numbers. I fixed the logic, backfilled clean data, and added a validation test so the same class of issue can’t repeat. The key learning: test with real edge cases, not just happy-path samples.

2) “Tell me about a time you missed a deadline.”

Your ready answer:
I underestimated dependencies on an upstream team and the data arrived later than expected. I communicated early with a revised ETA and a phased plan: deliver core fields first, then the remaining fields in the next drop. I also added a dependency checklist for future work (source readiness, schema sign-off, sample data). After that, timelines became more predictable.

3) “Tell me about a time you broke production.”

Your ready answer:
A release introduced a schema mismatch that failed a scheduled run. I triggered incident steps: stopped the blast radius, rolled back to the last stable version, and informed stakeholders with a clear ETA. Then I fixed the root cause and added schema checks +

DataGeeks

COPY RIGHTS RESERVED

deployment gating so incompatible changes can't get promoted again. I treat "break prod" as a process improvement moment, not an excuse story.

4) "Tell me about a mistake you made and what you learned."

Your ready answer:
I assumed a column would always be non-null, and I didn't handle nulls properly in joins. It caused unexpected record drops. I corrected it by adding safe null-handling and data quality checks (null rate, key uniqueness). My learning: always validate assumptions with profiling and make pipeline behavior explicit.

5) "Tell me about a time your work had a quality issue."

Your ready answer:
A dashboard number was off because of a metric definition mismatch, not the raw data. I set up a short session with the analyst/PM, finalized the definition (grain, inclusions/exclusions), and updated the model accordingly. I also documented the metric definition and added an automated check comparing totals across layers. After that, those "number mismatch" issues reduced a lot.

6) "Tell me about a time you made the wrong technical decision."

Your ready answer:
I initially chose a design that was fast to build but expensive to run at scale. When cost spiked, I took ownership, analyzed the hotspots, and redesigned: pushed filters earlier, reduced full scans, and improved partitioning/incremental logic. Result: lower compute usage and stable SLAs. Learning: always think about cost/perf from day one, not after production pain.

7) "Tell me about a time you had to roll back a change."

Your ready answer:
A change increased latency and impacted the SLA. I rolled back quickly using our release process, restored stability, and then reintroduced the fix behind a safer approach (feature toggle / phased rollout). I also updated runbook steps so rollback is fast and clean next time.

DataGeeks

COPY RIGHTS RESERVED

8) “Tell me about a time you received critical feedback.”

Your ready answer:
I got feedback that my updates were too technical and not clear for stakeholders. I adjusted by sending short status notes: what happened, impact, ETA, and next step. I also started adding one simple metric to every update. That improved trust and reduced repeated follow-up questions.

9) “Tell me about a time you took accountability when it wasn’t fully your fault.”

Your ready answer:
An upstream change impacted my pipeline, but from business side it still looked like “data is broken.” I owned the resolution: created a clear issue summary, coordinated with the upstream owner, and added a temporary guardrail to protect downstream. I don’t play blame games—I focus on restoring service and preventing repeats.

10) “Tell me about a time you handled a high-severity issue under pressure.”

Your ready answer:
We had a production issue affecting a critical report. I stayed calm, prioritized recovery first (restore last good run or rollback), kept stakeholders updated every fixed interval, and then did a proper root cause. After recovery, I implemented prevention: better alerts, stricter validations, and tighter release checks. The outcome was stable runs and fewer escalations.

1) “No one told you what to do what did you do?”

Your ready answer (say this):
When direction is unclear, you create clarity. I first confirm the business goal, then I map the current data flow, and list the biggest risks (freshness, quality, cost). I propose a simple plan with 2–3 milestones, get a quick sign-off, and start delivering in small releases with monitoring. That’s how you move without waiting.

2) “Requirements keep changing how did you handle it?”

DataGeeks

COPY RIGHTS RESERVED

Your ready answer:
I locked a baseline: “For this release, we’ll deliver A and B. C goes to phase 2.”
Then I asked for one owner to confirm changes, and I documented assumptions in a short note. When changes came, I showed impact on timeline and offered options. This kept the project calm and predictable.

3) “You have 3 priorities how do you choose?”

Your ready answer:
I rank by: **customer/business impact**, **risk**, and **deadline**.
If one item blocks others, it goes first. If something reduces production risk, it’s high priority. I also confirm with manager/PM in one message so priorities are visible and nobody feels ignored.

4) “You inherited a messy pipeline what is your plan?”

Your ready answer:
I follow a clean 3-step approach:

1. **Stabilize**: fix failures, add alerts, stop bleeding
2. **Understand**: data profiling, lineage, owners, SLAs
3. **Improve**: refactor logic, add tests/DQ checks, optimize performance and cost
   I don’t rewrite everything on day one—I make it reliable first.

5) “Tell me about a time you proactively improved something that wasn’t assigned to you.”

Your ready answer:
I noticed repeated incidents around a pipeline and it was wasting team time. I proposed a small improvement: better validations + alerting + a runbook. I implemented it with minimal disruption. Result: fewer failures and faster triage. I did it because it improved team productivity and reliability, not because someone told me.

6) “How do you handle ambiguity when stakeholders don’t know what they want?”

DataGeeks

COPY RIGHTS RESERVED

Your ready answer:
I use examples to force clarity:
“Show me one report or one row you expect.”
Then I define the metric and grain, and propose 2 options: a simple version vs a detailed version. Once they pick, I document the definition and proceed. This avoids building the wrong thing.

7) “Tell me about a time you took ownership end-to-end.”

Your ready answer:
I owned a pipeline delivery from requirement to production: clarified definitions, designed the approach, implemented transformations, validated with stakeholders, deployed via CI/CD, and set up monitoring + alerts. After release, I tracked the SLA and fixed issues fast. The key is I didn’t stop at “code is done”—I made sure it runs reliably.

8) “When you don’t have enough data to decide, what do you do?”

Your ready answer:
I run a quick spike: small experiment, sample data profiling, and a rough estimate of cost/runtime. Then I share findings and recommend the safest option. I don’t freeze—I reduce uncertainty with a small test and move forward.

9) “How do you deal with unclear ownership across teams?”

Your ready answer:
I make ownership explicit: “Upstream owns schema stability, analytics owns definitions, I own pipeline logic and reliability.”
If needed, I set a short sync, write the owners and next steps, and keep it in the ticket. This prevents “everyone thought someone else would handle it.”

10) “How do you show ownership when something breaks but it’s not your fault?”

Your ready answer:
From business side, it’s still *our* problem. I focus on restoring service first: add a guardrail, rollback if needed, and communicate status. In parallel, I coordinate with the upstream

DataGeeks

COPY RIGHTS RESERVED

owner for the permanent fix. I document root cause and prevention so it doesn't repeat. That's real ownership.

1) "Tell me about a time you collaborated with Data Science / ML teams."

Your ready answer (say this):
I aligned on what the model needs: feature definitions, training vs inference data, refresh frequency, and backfill rules. I built a reliable pipeline with versioned datasets, added validation checks, and ensured reproducibility. Result: DS team spent less time cleaning data and more time improving the model.

2) "How do you work with backend engineers when data depends on APIs/events?"

Your ready answer:
I treat it like a contract. I confirm schema, event keys, and change process. I ask for sample payloads, error codes, and retry behavior. I also request a heads-up for schema changes. This avoids surprises and keeps pipelines stable.

3) "Tell me about a time you worked with QA / release teams."

Your ready answer:
I aligned on test scenarios early: edge cases, rollback plan, and validation queries. I provided a simple checklist for release (data counts, freshness, key checks). During release, I stayed available to verify outputs and fix quickly if any mismatch appears.

4) "How do you handle misalignment between Analyst numbers vs your pipeline numbers?"

Your ready answer:
I don't argue. I compare layer by layer: source → staging → curated → dashboard. Most mismatches come from definition or grain issues. I lock a single definition, document it, and add a test so the same mismatch doesn't repeat.

5) "How do you give feedback to a teammate without creating tension?"

DataGeeks

COPY RIGHTS RESERVED

Your ready answer:
I focus on the work, not the person. I say what I observed, why it matters (bug/risk/performance), and suggest a better approach. I also call out what they did well. The goal is improvement, not winning.

6) "How do you receive feedback when someone challenges your design?"

Your ready answer:
I don't take it personally. I ask clarifying questions, understand their concern, and validate with evidence (perf numbers, failure modes, cost). If they're right, I adapt fast. If there's a tradeoff, I document the decision so the team moves forward.

7) "Tell me about a time you helped a teammate unblock quickly."

Your ready answer:
A teammate was stuck due to data inconsistency and unclear join keys. I paired for 20 minutes, traced lineage, identified the correct keys and null patterns, and gave them a clean query/test to validate. They shipped the fix same day, and we updated the doc to prevent repeats.

8) "How do you collaborate across time zones or distributed teams?"

Your ready answer:
I use strong async habits: clear tickets, short design notes, and status updates with ETA and blockers. I keep meeting time minimal and decisions written down. That way progress doesn't depend on live calls.

9) "How do you handle a situation where another team is slow or unresponsive?"

Your ready answer:
I escalate politely with structure: show impact, provide examples, and give a clear ask with a deadline. Meanwhile, I build a temporary guardrail so my downstream isn't blocked. I keep communication calm and documented.

10) "What does 'good team culture' look like to you as a Data Engineer?"

DataGeeks

COPY RIGHTS RESERVED

Your ready answer:
Shared ownership, clean reviews, and no blame culture during incidents. People communicate early, document decisions, and support each other during on-call. The team improves systems, not just fixes symptoms.

1) "Tell me about a time you led something without being a manager."

Your ready answer:
I noticed repeated pipeline failures were wasting time. I created a small plan: fix root cause, add DQ checks, set alerts, and document a runbook. I aligned with the analyst and upstream owner, delivered in two small releases, and stabilized the pipeline. Result: fewer failures and less firefighting.

2) "How do you drive standards like CI/CD or data quality in a team?"

Your ready answer:
I start with a simple baseline checklist (tests, naming, rollback, alerts). I make templates so it's easy to follow. Then I enforce via PR checks, not manual policing. Over time, the team ships faster with fewer production issues.

Stakeholders (2)

3) "Tell me about a time you handled a difficult stakeholder."

Your ready answer:
A stakeholder wanted fast delivery but kept adding scope. I aligned on the top priority, gave two options with timelines, and asked them to choose. I delivered MVP first with guardrails, then phased the rest. Result: expectations stayed clear and trust improved.

4) "How do you explain a delay to business users?"

Your ready answer:
I share facts + plan + ETA: what happened, business impact, what I'm doing, and when it will be fixed. I avoid jargon. I also give the next update time so they don't chase me.

Conflict (2)

5) "Tell me about a time you disagreed with a teammate on design."

DataGeeks

COPY RIGHTS RESERVED

Your ready answer:
We had two approaches. I laid out tradeoffs (risk, cost, maintainability) and proposed a small test. We picked the safer approach for production and documented the decision. Result: no rollback and better alignment.

6) "How do you push back on an unrealistic deadline?"

Your ready answer:
I offer options: MVP by date A with basic checks, or full solution by date B with stronger testing and monitoring. I let them choose based on business priority. That's how I push back without conflict.

Failure (2)

7) "Tell me about a time you made a mistake."

Your ready answer:
I missed an edge case in join logic and it dropped records. I owned it, fixed logic, backfilled, and added a test + DQ check to prevent repeats. Learning: profile data and validate assumptions early.

8) "Tell me about a time you broke production."

Your ready answer:
A deployment caused a pipeline failure. I rolled back to stabilize, communicated status, fixed the root cause, and added schema validation + release gating. Result: fast recovery and stronger prevention.

Ownership + Teamwork (2)

9) "Tell me about a time you improved something that wasn't assigned to you."

Your ready answer:
I saw recurring incidents around a job. I added alerting, better error messages, and a runbook. It reduced triage time and improved reliability. I did it because it helped the whole team.

10) "Tell me about a time you collaborated across teams to deliver."

Your ready answer:
I coordinated with upstream engineers for schema contract, analyst for metric definition,

DataGeeks

COPY RIGHTS RESERVED

and QA for validation checks. I delivered in phases and kept everyone updated. Result: smooth release and fewer post-release issues.

11) “You joined a project mid-way and nothing is documented. What do you do first?”

Your ready answer:
First, you stabilize understanding: you map the data flow (source → staging → curated → reports), list owners, and identify SLAs. Then you pick the biggest risk (freshness/quality/failures) and fix that first. You document as you go—one page with definitions, lineage, and runbook—so the next person doesn’t struggle.

12) “No one is owning a recurring issue. What do you do?”

Your ready answer:
You take temporary ownership to stop the pain. You quantify impact, create a clear owner map (upstream, pipeline, dashboard), and drive a short action plan. You fix what you can on your side (checks/alerts/retries) and align other owners for permanent fixes. The goal is reliability, not blame.

13) “Requirements are unclear. How do you move forward without wasting time?”

Your ready answer:
You ask for one example output and define the metric + grain. You write assumptions in 3 lines and get quick confirmation. Then you ship an MVP with guardrails and iterate. This way you deliver value fast without building the wrong thing.

Stakeholders + Communication (3)

14) “A business user says: ‘Your numbers are wrong.’ How do you respond?”

Your ready answer:
You respond calmly and go layer by layer: source → transformation → final table → dashboard. Most issues are definition/grain mismatch. You align on one definition, fix logic if needed, and add a check so it doesn’t repeat. You keep the user updated with ETA and what changed.

15) “How do you say ‘no’ to a request without sounding unhelpful?”

Your ready answer:
You don’t say no directly. You say: “Yes, we can do it—here are the tradeoffs.” Then you give

DataGeeks

COPY RIGHTS RESERVED

options: deliver core now, rest in phase 2; or push the date for full scope. People accept pushback when you give a clear path.

16) "How do you explain a complex pipeline to a non-technical person?"

Your ready answer:
You explain outcomes first: what data comes in, what it becomes, and when it's ready. Then you add one simple control: quality checks and monitoring. If they want details, you mention tools (ADF/Databricks/dbt/Snowflake) only after the business flow is clear.

Teamwork + Collaboration (2)

17) "Tell me about a time you worked with an upstream team on schema changes."

Your ready answer:
You aligned on a schema contract: keys, null rules, and change notification. When a change came, you protected downstream with validation and backward-compatible handling where possible. You also documented the contract so future changes don't break production silently.

18) "Tell me about a time you handled a disagreement with QA/release team."

Your ready answer:
You aligned on what 'done' means: validation queries, edge cases, rollback plan. Instead of debating, you used evidence—counts, freshness checks, and sample reconciliation—to reach a decision. Result: smoother release and fewer post-release escalations.

Learning Mindset + Growth (2)

19) "Tell me about a time you had to learn something quickly to deliver."

Your ready answer:
You learned just enough to solve the problem: you read docs, tested with a small POC, and applied it to production safely. You didn't experiment in production—you validated first, then rolled out with monitoring. Result: delivered on time and built confidence in the new approach.

20) "How do you keep your skills updated as a Data Engineer?"

Your ready answer:
You learn through real problems: pick one area (cost, reliability, modeling, streaming),

DataGeeks

COPY RIGHTS RESERVED

apply it in your work, and document learnings. You also do short weekly practice—review one incident, one optimization, one design pattern. That keeps you sharp without random learning.

Incident / On-call / Reliability (4)

21) "Tell me about a time you handled an on-call incident."

Your ready answer:
A critical pipeline failed and impacted a reporting SLA. I first contained impact: paused downstream refresh, ran a safe fallback, and informed stakeholders with ETA. Then I fixed root cause, replayed/backfilled data, and added prevention (alerting, retries, validation). Result: faster recovery next time and fewer repeated incidents.

22) "How do you communicate during an outage?"

Your ready answer:
I send short updates: **what's impacted**, **what I'm doing**, **ETA**, and **next update time**. I avoid deep tech details unless asked. The goal is to reduce panic and keep everyone aligned.

23) "Tell me about a time you improved reliability of a pipeline."

Your ready answer:
I saw recurring failures due to upstream data issues. I added validation checks, idempotent reruns, better error logs, and alerts tied to SLA. I also created a runbook so anyone can triage. Result: failures reduced and triage time dropped.

24) "Tell me about a time you prevented an incident before it happened."

Your ready answer:
I noticed early warning signs—latency trending up and null rates increasing. I paused the release, validated the change, and fixed the issue before it hit production. I also added monitoring thresholds so the team gets notified before SLAs break.

Leadership / Influence (3)

25) "Tell me about a time you influenced a decision with data."

Your ready answer:
We had two design options. I compared them using evidence: estimated runtime, cost,

DataGeeks

COPY RIGHTS RESERVED

failure risk, and maintenance effort. I shared the tradeoff clearly and recommended the safer long-term option. The team aligned, and we avoided future rework.

26) "Tell me about a time you improved a team process."

Your ready answer:
Our releases were messy and caused last-minute issues. I introduced a simple release checklist: tests, validation queries, monitoring, and rollback steps. It made releases predictable, reduced production surprises, and improved team confidence.

27) "Tell me about a time you mentored someone or raised the bar."

Your ready answer:
I helped a junior engineer by pairing on one pipeline, teaching how to handle edge cases, and reviewing PRs with clear guidance. I also shared a small template for logging/testing. Over a few sprints, their work quality improved and review cycles became faster.

Conflict / Pushback (3)

28) "Tell me about a time you disagreed with a stakeholder's request."

Your ready answer:
A stakeholder wanted extra scope in the same timeline. I explained impact and gave options: MVP now vs full scope later. We agreed on phased delivery and I documented the decision. Result: no surprises and better trust.

29) "Tell me about a time you had to say 'this is risky' to a team."

Your ready answer:
A proposed change could break downstream consumers. I flagged the risk with examples, suggested a safer rollout (backward compatible change / feature flag), and proposed validation checks. The team accepted the safer approach and we avoided a rollback.

30) "Tell me about a time you resolved a conflict between two teams."

Your ready answer:
Two teams had different definitions and were blaming each other. I brought them together, walked through lineage, agreed on one definition and ownership, and captured it in a shared doc. Then we added a validation test so the mismatch doesn't return.

31) "Tell me about a time you delivered under tight deadline."

COPY RIGHTS RESERVED

Your ready answer:
I split it into **MVP + hardening**. I delivered the core pipeline first with basic DQ checks, retries, and monitoring. I aligned stakeholders on what's in/out for release 1. Then I planned performance tuning and extra validations in the next sprint. Result: on-time delivery without risky shortcuts.

32) "Tell me about a time you had to manage multiple urgent asks."

Your ready answer:
I ranked tasks by **business impact + SLA risk + blockers**. I confirmed priorities with my manager/PM in one message to avoid confusion. I delivered quick wins first (stability fixes), then moved to feature work. Result: fewer escalations and predictable progress.

33) "Tell me about a time you had to reset expectations."

Your ready answer:
I noticed scope and timeline were drifting. I shared a simple update: what changed, impact on ETA, and 2 options (reduce scope vs extend timeline). I got a clear decision and updated the plan. Result: no surprise delays and stronger trust.

Migration / Legacy modernization (3)

34) "Tell me about a migration you worked on (legacy → modern)."

Your ready answer:
I migrated a pipeline by running **parallel pipelines** first, validating outputs with reconciliation checks, and switching traffic only after confidence. I handled schema differences and ensured rollback was ready. Result: smooth cutover, stable SLAs, and reduced operational load.

35) "How do you approach migrating without downtime?"

Your ready answer:
You do it in phases: baseline current outputs, build new path, run in parallel, compare results, and cut over behind a controlled switch. You keep rollback simple. You also communicate clearly: when comparison starts, when cutover happens, and success criteria.

36) "Tell me about a time you handled a big backfill during migration."

Your ready answer:
I planned backfill with throttling and checkpoints so it wouldn't overload compute. I

DataGeeks

COPY RIGHTS RESERVED

validated in batches and tracked progress with run logs. I also aligned consumers on when historical data becomes consistent. Result: backfill completed safely without breaking daily SLAs.

Cost / Performance optimization (4)

37) "Tell me about a time cloud cost spiked. What did you do?"

Your ready answer:
I first identified the biggest cost driver (jobs, warehouses, scans). Then I fixed root causes: reduced full scans, improved filtering early, and optimized incremental logic. I added cost monitoring alerts so spikes get caught early. Result: cost stabilized and spend reduced.

38) "Tell me about a time you improved pipeline runtime."

Your ready answer:
I profiled the slow steps, then optimized joins, partitioning/clustering, and reduced shuffles/full table reads. I also removed unnecessary columns and pushed filters earlier. Result: faster runs and fewer SLA misses, with proof via before/after runtime metrics.

39) "Tell me about a time you optimized a slow SQL model / dbt model."

Your ready answer:
I checked query plan patterns: large joins, missing keys, and non-selective filters. Then I rewrote logic to reduce scans, used incremental builds where possible, and validated results with tests. Result: lower warehouse time and more predictable model runs.

40) "Tell me about a time you made a tradeoff between cost and freshness."

Your ready answer:
A stakeholder wanted near real-time, but cost would jump a lot. I proposed a middle option: frequent micro-batches during peak hours and hourly outside peak, with clear SLAs. We met business needs without constant high compute. Result: balanced freshness and spend.

41) "Tell me about a time you found a data quality issue before the business did."

Your ready answer:
I noticed an unusual spike in nulls/duplicates in a key dataset through monitoring. I paused downstream refresh, traced the issue to an upstream change, and applied a guardrail

DataGeeks

COPY RIGHTS RESERVED

(validation + quarantine). Then I coordinated the permanent fix with the source owner. Result: business never saw wrong numbers, and we prevented repeat issues.

42) “Tell me about a time business lost trust in data. What did you do?”

Your ready answer:
I first acknowledged the impact and stopped guessing. I did a full reconciliation across layers, aligned on one metric definition, and fixed the pipeline/model. Then I added clear SLAs, DQ checks, and a simple definitions page. Result: fewer ‘number mismatch’ pings and trust came back slowly but strongly.

43) “How do you ensure data quality without slowing delivery?”

Your ready answer:
I bake in lightweight checks: null checks, uniqueness on keys, freshness checks, and row-count anomaly alerts. These are fast, automated, and catch 80% issues. For deep checks, I add them later as phase 2. That keeps delivery fast but safe.

44) “Tell me about a time you aligned metric definitions across teams.”

Your ready answer:
Different teams had different definitions of the same KPI. I ran a short session, wrote the definition in plain words (grain + inclusions/exclusions), and got sign-off from analyst/PM. Then I updated the model and added tests to enforce it. Result: one source of truth and fewer debates.

Schema changes / Breaking changes (3)

45) “Tell me about a time a schema change broke your pipeline.”

Your ready answer:
An upstream schema change caused a job failure. I restored service first (rollback/hotfix), then updated the pipeline to handle the new schema safely. I added schema validation and a change-notification process with upstream. Result: faster recovery and fewer surprise breaks.

46) “How do you prevent schema change incidents long-term?”

Your ready answer:
You treat it like a contract: versioned schemas, backward-compatible changes where possible, automated checks in CI/CD, and early alerts when schema drifts. Also, clear

DataGeeks

COPY RIGHTS RESERVED

ownership: upstream must notify changes, and downstream must validate before deploying.

47) “Tell me about a time you handled backward compatibility for downstream consumers.”

Your ready answer:
A new field or rename could break dashboards. I kept old columns for a transition window, added a new column in parallel, and communicated a deprecation date. I monitored consumers and removed the old path only after adoption. Result: smooth transition with no outages.

Culture / Feedback / Working style (3)

48) “Tell me about a time you gave tough feedback to a teammate.”

Your ready answer:
I kept it factual and private. I explained what I observed, why it’s risky (bug/perf/maintainability), and what good looks like. I offered help and a simple checklist. Result: they improved quickly and the relationship stayed healthy.

49) “Tell me about a time you received feedback and changed your approach.”

Your ready answer:
I got feedback that my updates were too detailed and hard to follow. I switched to short updates: impact, ETA, next step, and one metric. It reduced confusion and made stakeholders feel informed without extra meetings.

50) “How do you build good relationships with cross-functional partners?”

Your ready answer:
I communicate early, set clear expectations (SLA/quality), and avoid surprises. When something breaks, I focus on resolution, not blame. Over time, people trust you because you’re reliable and transparent.

51) “Tell me about a time you had to learn a new tool fast to deliver.”

Your ready answer:
I focused on the exact gap needed for delivery. I read the basics, built a small POC with sample data, validated outputs, then applied it to the real pipeline with monitoring. I didn’t over-learn. Result: delivered on time and gained a reusable pattern for future work.

DataGeeks

COPY RIGHTS RESERVED

52) “How do you upskill without burning out?”

Your ready answer:
I keep it practical: 20–30 minutes a day on one topic tied to my current work (performance, cost, DQ, modeling). I learn, apply once, and document the pattern. Small consistency beats weekend cramming.

53) “Tell me about a time you asked for help.”

Your ready answer:
I got stuck on a tricky data mismatch. I first narrowed it down with evidence (sample rows, counts), then asked the right SME with a clear question. I also shared what I already tried. Result: faster resolution and better collaboration.

54) “Tell me about a time you improved after a failure.”

Your ready answer:
After a production issue, I didn’t stop at the fix. I added prevention: tests, validation, monitoring, and release gates. I also updated the runbook. Result: the same issue never repeated, and the team’s on-call load reduced.

Ownership under ambiguity (3)

55) “You don’t have complete requirements, but business wants progress. What do you do?”

Your ready answer:
I deliver a thin slice: one core metric end-to-end with clear assumptions. I get feedback early, then expand scope. This shows progress, reduces rework, and keeps stakeholders engaged.

56) “Tell me about a time you drove clarity in a chaotic project.”

Your ready answer:
I created a simple alignment doc: goal, scope, owners, SLA, timeline, and risks. Then I ran a 15-minute alignment call to lock it. Result: fewer random requests and clearer decision-making.

57) “Tell me about a time you handled changing priorities mid-sprint.”

Your ready answer:
I re-ranked tasks based on impact and deadlines, confirmed priorities with my

DataGeeks

COPY RIGHTS RESERVED

manager/PM, and communicated what moved out. I kept stakeholders informed with new ETA. Result: no confusion and fewer last-minute escalations.

Leadership at scale / Systems thinking (3)

58) "How do you design pipelines so they're easy to operate at scale?"

Your ready answer:
I design for failure: idempotent runs, retries, clear logging, alerts, and runbooks. I also keep interfaces stable (contracts) and add tests for key metrics. That's what makes pipelines reliable when volume and teams grow.

59) "Tell me about a time you improved observability or monitoring."

Your ready answer:
I added monitoring for freshness, row-count anomalies, failure reasons, and latency. I tied alerts to SLAs and created a simple dashboard for pipeline health. Result: issues were caught earlier and triage became faster.

60) "Tell me about a time you improved a process beyond your own work."

Your ready answer:
I noticed repeated review comments and inconsistent patterns. I created a small template/checklist for pipelines (naming, tests, DQ checks, release steps) and shared it with the team. Result: faster PR cycles and fewer production issues.

Stakeholder escalation / Tough conversations (4)

61) "Tell me about a time you handled an escalation from business."

Your ready answer:
A dashboard was showing unexpected numbers and stakeholders escalated. I acknowledged impact, paused further refresh if needed, and shared a clear ETA. I traced layer by layer, found the root cause (definition or upstream change), fixed it, and backfilled. Then I added a validation check and a short metric definition note. Result: issue closed with prevention.

62) "How do you handle a stakeholder who keeps pinging you every hour?"

Your ready answer:
I reduce uncertainty. I give a fixed update rhythm: "I'll update you at 1 PM and 4 PM." I also

DataGeeks

COPY RIGHTS RESERVED

share what's blocked and what I'm doing next. Most people stop chasing when you give a predictable communication loop.

63) "Tell me about a time you had to deliver bad news."

Your ready answer:
I shared it early with facts: what happened, impact, and realistic ETA. I offered a fallback plan (partial data, last good snapshot, or limited scope). I also explained what I'm doing to prevent repeats. Result: less frustration because there were no surprises.

64) "Tell me about a time you changed a stakeholder's mind."

Your ready answer:
They wanted a quick solution that was risky long-term. I presented two options with tradeoffs and backed it with evidence (cost, runtime, failure risk). We chose the safer approach, shipped with phased rollout, and avoided future rework.

Conflict with manager / Priorities (3)

65) "Tell me about a time you disagreed with your manager."

Your ready answer:
I disagreed on priority because it increased SLA risk elsewhere. I explained the impact clearly, suggested an alternate order, and asked for a decision. Once decided, I committed fully and updated stakeholders on the plan. Result: alignment without friction.

66) "Your manager asks you to 'just ship it' but you think it's risky. What do you do?"

Your ready answer:
I frame risk in business terms: "If we ship now, we might break downstream reports and cause rollback." I propose a minimal safety step: add validation, staged rollout, or feature flag. If they still want ship, I document the decision and ensure rollback is ready.

67) "Tell me about a time you handled competing priorities from multiple leaders."

Your ready answer:
I made priorities visible. I listed tasks with impact and deadlines, highlighted dependencies, and asked for one final ranking in writing. Then I executed based on that list and sent weekly progress updates. Result: fewer conflicting asks and clear accountability.

Tradeoffs / Decision making (3)

COPY RIGHTS RESERVED

68) “Tell me about a time you had to choose between correctness and speed.”

Your ready answer:
We needed quick insights, but full accuracy would take longer. I delivered a fast version with clear limitations and guardrails, and planned a second phase for full correctness. I communicated what’s included/excluded. Result: business got value quickly without being misled.

69) “Tell me about a time you chose simplicity over a ‘perfect’ design.”

Your ready answer:
A complex design would have delayed delivery and been harder to maintain. I chose a simpler approach that met requirements, added monitoring and tests, and kept it extensible for future changes. Result: shipped faster and reduced operational complexity.

70) “Tell me about a time you took a decision with incomplete information.”

Your ready answer:
I ran a quick spike: sample data profiling + small POC + rough cost/runtime estimate. Then I proposed the safest option with clear assumptions and a rollback plan. Result: moved forward confidently without waiting endlessly.

Team culture / Collaboration habits (4)

71) “What kind of team culture do you thrive in?”

Your ready answer:
I do best in a culture with clear ownership, strong reviews, and no blame during incidents. People communicate early, document decisions, and support each other. I like teams that value reliability and continuous improvement, not just shipping fast.

72) “Tell me about a time you improved collaboration in the team.”

Your ready answer:
There was confusion between upstream, pipeline, and dashboard owners. I created a simple ownership map and a shared doc for definitions + SLAs. We reduced random pings and issues got resolved faster because everyone knew who owns what.

73) “How do you handle disagreements in the team without affecting relationships?”

Your ready answer:
I keep it factual: align on goal, share evidence, propose options, and document the

DataGeeks

COPY RIGHTS RESERVED

decision. I avoid personal comments. Even if we disagree, we move forward as one team after the decision is made.

74) “Tell me about a time you supported a teammate during pressure.”

Your ready answer:
A teammate was overloaded close to release. I took one slice end-to-end (validation + rollout checks), and I paired to unblock them quickly. We shipped on time, and it also built trust because the support was practical.

Ownership habits / Working style (3)

75) “How do you ensure your work is reliable before deploying to production?”

Your ready answer:
I validate with sample and edge cases, run reconciliation checks, and confirm SLAs. I ensure monitoring and rollback steps are ready. I don’t treat deployment as the end—production stability is part of done.

76) “How do you prioritize when everything seems urgent?”

Your ready answer:
I prioritize by business impact, SLA risk, and blockers. I confirm priority order with my manager/PM quickly. Then I communicate what I’m doing first and what moves out. That reduces confusion and escalations.

77) “Tell me about a time you owned something end-to-end.”

Your ready answer:
I took a data pipeline from requirement to production: clarified definitions, designed the logic, built transformations, validated with stakeholders, deployed through CI/CD, and set up monitoring and alerts. After release, I tracked SLA and handled fixes quickly.

Closing / Self-awareness / Fit (3)

78) “What’s your biggest weakness as a Data Engineer?”

Your ready answer:
Earlier I used to go too deep into technical details in updates. I fixed that by using short, clear communication: impact, ETA, next step, and one metric. It improved stakeholder trust and reduced repeated questions.

DataGeeks

COPY RIGHTS RESERVED

79) “Why should we hire you?”

Your ready answer:
Because I deliver reliable pipelines with clear ownership. I focus on outcomes: quality, freshness, performance, and cost. I communicate well with stakeholders, handle incidents calmly, and I leave systems better than I found them—monitoring, tests, and documentation included.

80) “Tell me about yourself (behavioral-style, not a long intro).”

Your ready answer:
I’m a data engineer who focuses on building reliable, scalable pipelines that teams can trust. I’ve worked on batch and large-scale transformations, and I care a lot about data quality, monitoring, and smooth deployments. I work well with analysts, engineers, and stakeholders because I explain clearly and I take ownership end-to-end.

1) Incident / on-call outage (pipeline failed, SLA breach)

Your ready answer (say this):
“An important production pipeline failed and the business report SLA was at risk. First, you stop the blast radius—pause downstream refresh, check the last successful run, and decide rollback vs hotfix. You communicate early: impact, ETA, next update time. Then you fix the root cause (schema drift, bad file, infra issue), rerun idempotently, and backfill missing partitions. After recovery, you do prevention: add alerts on freshness + failure, add validation checks, and update the runbook so next time recovery is faster. Result: SLA restored, fewer repeated incidents, and clearer operational ownership.”

2) Data quality issue in dashboard (numbers mismatch)

Your ready answer:
“A stakeholder says the dashboard number is wrong. You don’t argue—you reconcile. You trace source → staging → curated → dashboard and check grain + filters + definitions. Most times it’s definition mismatch or join duplication. You align on one definition with analyst/PM, fix the transformation, and backfill the corrected window. You also add a DQ test (uniqueness, nulls, row-count anomalies) and a ‘metric definition’ note so the same confusion doesn’t come back. Result: numbers match, trust improves, and future mismatches reduce.”

DataGeeks

COPY RIGHTS RESERVED

3) Schema change broke pipeline (upstream changed column/type)

Your ready answer:
"An upstream schema change broke your pipeline run. First, you restore service: rollback or quick patch, so business isn't blocked. Then you make it resilient: add schema validation at ingestion, handle optional columns safely, and enforce a change notification process with upstream owners. You document the data contract (keys, null rules, allowed changes). Result: faster recovery and fewer surprise breaks going forward."

4) Cost spike in cloud (compute spend suddenly increased)

Your ready answer:
"Cost spiked unexpectedly. You start with evidence: which job/warehouse/cluster increased and what changed (schedule, full scan, data growth). Then you fix root cause: push filters earlier, move to incremental logic, reduce shuffles/full scans, and tune partitioning/clustering. You also add cost monitoring alerts and a weekly cost review for top offenders. Result: spend stabilizes and you can show a before/after cost drop with the same SLA."

5) Migration under deadline (legacy → modern platform)

Your ready answer:
"You had to migrate quickly without breaking consumers. You do it in phases: baseline current outputs, build the new pipeline/model, run in parallel, and reconcile results with automated checks. Only after confidence, you cut over with a rollback plan ready. You keep stakeholders updated with clear success criteria (freshness, counts, key totals). Result: safe cutover, minimal downtime, and reduced operational pain."

6) Stakeholder wants impossible timeline (scope creep + pressure)

Your ready answer:
"They wanted full scope in an unrealistic time. You don't say 'no'—you give options. Option A: MVP by Friday with core fields + basic DQ checks. Option B: full scope by next week with stronger validation, backfill, and monitoring. You ask them to choose what matters most. You document scope and ETA in the ticket so it's clean. Result: expectations managed, delivery stays calm, and you protect quality."

DataGeeks

COPY RIGHTS RESERVED

7) Team disagreement on design (two approaches, high risk)

Your ready answer:
"The team had two design options. You align on the goal first (SLA, maintainability, cost). Then you compare tradeoffs with evidence: failure modes, performance, complexity, and long-term support. If needed, you propose a small spike/POC to de-risk. After decision, you document it in PR/design notes so it doesn't re-open. Result: faster alignment, fewer rework loops, and stronger engineering maturity."

8) Mentoring junior engineer (quality + growth)

Your ready answer:
"A junior engineer was struggling with pipeline logic and edge cases. You paired once, explained the 'why', and gave them a small piece to own end-to-end. In PR reviews, you gave clear comments: correctness, performance, reliability, readability. You also shared a simple checklist/template for tests, logging, and DQ checks. Result: their PR quality improved, review cycles reduced, and the team shipped faster."

9) Handling production rollback (bad release, restore fast)

Your ready answer:
"A release caused unexpected failures/latency. You rolled back to last stable version quickly to restore service. You communicated status and ETA. Then you reproduced the issue in lower environment, fixed it properly, and re-released using a safer rollout (feature flag / phased deployment / extra validation). You updated the runbook and added release gating tests. Result: minimal downtime and stronger release discipline."

10) Driving CI/CD adoption (team used manual deployments)

Your ready answer:
"Deployments were manual and risky. You started small: a basic pipeline that runs lint/tests, validates SQL/dbt models, and enforces PR checks. You made templates so adoption is easy, added rollback steps, and kept the process lightweight. You also showed quick wins fewer production issues, faster releases, clearer audit trail. Result: the team moved from manual deployments to repeatable, safer releases with less stress."

DataGeeks

COPY RIGHTS RESERVED

1) Hiring Manager: "What does success look like in the first 30/60/90 days for this role?"

Why you ask: You're showing you care about outcomes, not just tasks.
Good answer you want to hear: Clear milestones like "own X pipeline, reduce failures, ship Y model, improve SLA."
If they're vague, your follow-up: "What are the top 2 problems you want this person to solve first?"

2) Hiring Manager: "What are the biggest data reliability issues today—freshness, quality, or cost?"

Why you ask: It signals you think like an operator.
Good answer you want: They can name 1–2 real pain points and how they measure them.
Follow-up: "How do you track this dashboards, alerts, weekly reviews?"

3) Hiring Manager: "What's the current expectation around on-call and incident response?"

Why you ask: You're being practical and mature.
Good answer you want: Rotation details, severity process, postmortems, ownership clarity.
Follow-up: "When an incident happens, who leads data engineering or platform?"

4) Hiring Manager: "How do you balance speed vs data quality in releases?"

Why you ask: Product companies love tradeoff thinking.
Good answer you want: MVP + guardrails, tests, DQ checks, phased rollouts.
Follow-up: "What are 'must-have' quality checks before production?"

5) Data Team: "How do you define and enforce metric definitions so teams don't fight over numbers?"

Why you ask: You're hitting the most common real-world failure.
Good answer you want: Single source of truth, definitions doc, dbt tests/semantic layer, governance.
Follow-up: "Who owns metric definitions analytics, product, or data engineering?"

DataGeeks

COPY RIGHTS RESERVED

6) Data Team: “Where do you see the biggest bottleneck today ingestion, modeling, orchestration, or consumption?”

Why you ask: You’re mapping where you’ll create impact fast.
Good answer you want: Specific bottlenecks + what they tried already.
Follow-up: “If I join, what’s the one bottleneck you’d want me to attack first?”

7) Data Team: “How do code reviews work here for SQL/dbt/PySpark what’s ‘good’?”

Why you ask: You’re showing you care about standards and maintainability.
Good answer you want: Review checklist (correctness, performance, tests, naming, docs).
Follow-up: “Do you have a template or automated checks in CI?”

8) Cross-functional: “How do PM/Analysts typically request work tickets, docs, or ad-hoc?”

Why you ask: You’re protecting delivery from chaos.
Good answer you want: Clear intake process, prioritization, acceptance criteria.
Follow-up: “What’s the fastest way to get clarity when requirements are vague?”

9) Cross-functional: “What’s the most common reason for last-minute changes or scope creep?”

Why you ask: You’re quietly checking team maturity.
Good answer you want: Honest answer + how they handle it (phased releases, change control).
Follow-up: “How do you freeze scope for a release while still staying flexible?”

10) Cross-functional: “When data issues happen, what does communication look like with stakeholders?”

Why you ask: You’re showing leadership in expectation management.
Good answer you want: Update cadence, shared incident channel, owner + ETA style

DataGeeks

COPY RIGHTS RESERVED

comms.
Follow-up: “Do stakeholders prefer frequent short updates or one final update?”

Closing line that leaves impact (say this at the end)

“Thanks this helps a lot. Based on what you shared, I can see I’ll add value by improving reliability and making delivery predictable. If we move forward, I’d love to align on the first 30-day success goals and start owning one key pipeline end-to-end.”

1) “What do you do in the first 5 minutes before the interview?”

Your ready answer (say this):
You refresh your **top 3 stories**: one win, one failure, one conflict. You don’t read everything. You just recall the **one-liner → 3 bullets → metric** for each. That’s enough to sound confident from minute one.

2) “How do you pick the ‘top 3 stories’ for this specific company?”

Your ready answer:
You match the JD to stories:

- If they mention reliability/on-call → pick your incident story
- If they mention stakeholder + metrics → pick your data quality / definition story
- If they mention scale/cost → pick your optimization story
  Same stories, different spotlight.

3) “What do you do in the next 10 minutes for company alignment?”

Your ready answer:
You learn 3 things:

1. What the company does (in one line)
2. What data likely matters (orders, users, revenue, churn, etc.)
3. What risks matter (freshness, quality, cost, latency)
   Then you prepare one line: “I can help you improve X with Y.”

DataGeeks

COPY RIGHTS RESERVED

4) “How do you do role alignment in 10 minutes?”

Your ready answer:
You pick 3 keywords from the JD (ex: dbt, Snowflake, streaming, governance). For each, you prepare **one proof line** from your experience: what you built, what improved, and one metric. This makes you sound instantly relevant.

5) “How do you prepare for ‘Tell me about yourself’ in 60 seconds?”

Your ready answer:
You use this structure:

- 1 line: who you are (data engineer focused on reliability/scale)
- 1 line: what you’ve been doing recently (pipelines, modeling, optimization)
- 1 line: proof (metric/result)
- 1 line: why this role (match JD)
  Stop. Don’t add your whole life story.

6) “How do you prepare for follow-up questions (deep dives)?”

Your ready answer:
You prepare 3 follow-up layers for each story:

- What tradeoff you made
- How you validated correctness
- How you monitored after release
  These are the most common follow-ups in product companies.

7) “What is your 5-minute ‘closing pitch’ preparation?”

Your ready answer:
You prepare a closing line with value:
“From what I understood, your key problems are X and Y. I’ve solved similar things by doing

DataGeeks

COPY RIGHTS RESERVED

A and B, and I'd bring the same ownership here."
Then you ask 1 smart question from Section 12.

8) "What do you do if you feel nervous or blank in the interview?"

Your ready answer:
You buy 3 seconds politely:
"Let me think for a moment."
Then you fall back to your structure: one-liner → 3 bullets → metric. Structure reduces anxiety because you don't have to improvise.

9) "How do you avoid rambling in the actual interview?"

Your ready answer:
You finish your first answer in **under 90 seconds** and stop after the metric. If they want depth, they'll ask. If you keep talking, you reduce your score even if you're correct.

10) "What's the exact 30-minute breakdown you follow every time?"

Your ready answer:

- 5 mins: refresh top 3 stories
- 10 mins: company alignment (product + data + risks)
- 10 mins: role alignment (3 JD keywords → 3 proof lines)
- 5 mins: closing pitch + 1 sharp question
  That's it. Simple and repeatable.

1) "What is your 1-page STAR cheat sheet?"

Your ready answer:
STAR in 4 lines:

- **S (Situation):** what was happening + why it mattered
- **T (Task):** what you were responsible for
- **A (Action):** 3 key steps you personally did

DataGeeks

COPY RIGHTS RESERVED

- **R (Result):** metric + business effect + proof
  Rule: S+T ≤ 20 seconds, A ≤ 40 seconds, R ≤ 15 seconds.

2) "What is your 1-page CARE cheat sheet?"

Your ready answer:
CARE in 4 lines (faster than STAR):

- **C (Context):** short background + goal
- **A (Action):** what you did (3 bullets)
- **R (Result):** one metric + outcome
- **E (Evidence):** log/dashboard/cost report/postmortem
  Use CARE when the interviewer wants impact + proof quickly.

3) "What is your 'One-liner → 3 bullets → metric' cheat sheet?"

Your ready answer:
Template (copy/paste style):

- **One-liner:** "I solved ___ by ___."
- **3 bullets:** 1) ___ 2) ___ 3) ___
- **Metric:** "Result: ___ (time/cost/SLA/quality)."
  Rule: stop after the metric. Let them ask follow-ups.

4) "What's your 1-page 'top answers templates' cheat sheet?"

Your ready answer:
Most used templates:

- **Ownership:** "I took it end-to-end: clarify → build → validate → deploy → monitor."
- **Conflict:** "Align goal → facts → options → decision → document."
- **Failure:** "Own it → fix → prevent → learn."
- **Stakeholder pushback:** "Options A/B with timeline + tradeoffs."
- **Ambiguity:** "Assumptions + sample output + MVP + iterate."

DataGeeks

COPY RIGHTS RESERVED

5) “What’s your ‘red flags to avoid’ cheat sheet?”

Your ready answer:
Avoid these in behavioral rounds:

- Only tools, no outcomes (“I used Spark” with no impact)
- Blaming others (“upstream team issue”)
- No metric, no proof (“we improved a lot”)
- Long stories with no result
- Saying “we did” without your ownership
- Defending mistakes instead of owning them

6) “What are the 10 strongest metrics you can reuse anywhere?”

Your ready answer:
Pick 3–5 that match you:

1. Runtime reduced (X → Y)
2. Cost reduced (X% or ₹/$ per month)
3. SLA improved (T+1 → T+0 / 9 AM daily)
4. Failure rate reduced (daily → monthly)
5. Data quality improved (nulls/dupes down X%)
6. Backfill time improved
7. Incident recovery time improved
8. Adoption increased (teams/users using dataset)
9. Manual effort reduced (hours/week saved)
10. Rework reduced (fewer number mismatches/escalations)

7) “What is your ‘safe failure answer’ cheat sheet?”

DataGeeks

COPY RIGHTS RESERVED

Your ready answer:
Failure template:

- "I made a mistake in ___."
- "Impact was ___."
- "I fixed it by ___."
- "I prevented repeat by ___ (test/alert/gate/runbook)."
- "Now I always ___."
  Rule: no excuses, no blaming.

8) "What is your 'stakeholder communication' cheat sheet?"

Your ready answer:
Status update format (always):

- **Impact:** what is affected
- **Cause (simple):** why it happened
- **ETA:** when it will be fixed
- **Next update time:** when you'll update again
  **Keep it short. One metric if possible.

9) "What is your 'incident response' cheat sheet?"

Your ready answer:
Incident flow:

1. Contain impact (pause/rollback/fallback)
2. Communicate (impact, ETA, update cadence)
3. Fix root cause
4. Recover (rerun/backfill/replay)
5. Prevent (alerts, tests, gating, runbook)
   Rule: recovery first, analysis next, prevention last.

DataGeeks

COPY RIGHTS RESERVED

10) "What is your 1-page 'pre-interview checklist' cheat sheet?"

Your ready answer:
30-minute checklist:

- Top 3 stories ready (win, failure, conflict)
- 3 JD keywords → 3 proof lines
- 1 metric you'll repeat confidently
- 2 questions to ask interviewer
- Closing pitch: "I can help with X and Y by doing A and B."
  Rule: don't over-prepare. Be sharp and calm.

Cheat Sheet 15: Follow-up questions

1) Why did you choose this approach?

Ready line: "I chose it because it balanced reliability, cost, and delivery speed."

2) How did you validate correctness?

Ready line: "I validated with reconciliation checks: counts, key uniqueness, and sample record matching."

3) How did you handle edge cases?

Ready line: "I profiled data, handled nulls/duplicates, and tested worst-case scenarios."

4) How did you ensure it won't break again?

Ready line: "I added alerts, tests, and release gating so the same issue is caught early."

5) What was the biggest risk and how did you reduce it?

Ready line: "Schema drift / bad data / scale. I reduced it with contracts + validations + phased rollout."

6) How did you roll out safely?

Ready line: "I did a phased rollout, monitored runs, and kept rollback ready."

7) What tradeoff did you make?

DataGeeks

COPY RIGHTS RESERVED

Ready line: “We shipped MVP with guardrails first, then hardened performance and coverage next sprint.”

8) How did you communicate progress?

Ready line: “Short updates: impact, ETA, next step, next update time.”

9) What would you do differently now?

Ready line: “I’d add earlier profiling and stronger pre-prod validation on real data.”

10) How did you measure success?

Ready line: “One metric: SLA, failure rate, runtime, cost, or DQ improvement.”

Cheat Sheet 16: “What you say when you don’t know”

1) If you don’t know the exact answer

Ready line: “I haven’t done that exact thing, but I can walk you through how I’d approach it.”

2) If you need to clarify the question

Ready line: “Quick check—are you asking about correctness, performance, or operational reliability?”

3) If you’re missing requirements

Ready line: “Before deciding, I’d confirm the goal, the data grain, and the SLA.”

4) If it’s a new tool/tech

Ready line: “I’m not hands-on with this yet, but the core pattern is similar—ingest, validate, model, monitor.”

5) If you need time to think

Ready line: “Let me take a moment to structure this.”

6) If you’re unsure about an assumption

Ready line: “I don’t want to guess—here’s the assumption I’d verify first.”

7) If they ask something too broad

Ready line: “I’ll answer with a simple framework, then we can go deeper if you want.”

DataGeeks

COPY RIGHTS RESERVED

8) If you made a mistake earlier in the interview

Ready line: "I want to correct what I said—this is the accurate part."

9) If you've never faced that scenario

Ready line: "I haven't faced it directly, but I've handled similar risk patterns in production."

10) If they push you for a direct answer

Ready line: "My best answer is X, based on these constraints. If constraints change, I'd adjust to Y."

Top 10 Resources

1) Google re:Work – Structured interviewing guide

https://rework.withgoogle.com/intl/en/guides/hiring-use-structured-interviewing

2) MIT CAPD – STAR method for behavioral interviews

https://capd.mit.edu/resources/the-star-method-for-behavioral-interviews/

3) Amazon Jobs – Interview Loop (STAR + behavioral expectations)

https://amazon.jobs/content/en/how-we-hire/interview-loop

4) Interviewing.io – Amazon Leadership Principles guide

https://interviewing.io/guides/amazon-leadership-principles

5) Tech Interview Handbook – Behavioral interviews for software engineers

https://www.techinterviewhandbook.org/behavioral-interview/

6) TU Dublin – CARR Technique (PDF)

DataGeeks

COPY RIGHTS RESERVED

https://www.tudublin.ie/media/website/for-students/careers/docs/CARR-Technique-Help-Sheet.pdf

7) Exponent – Data Engineer behavioral questions

https://www.tryexponent.com/questions?role=data-engineer&type=behavioral

8) Pipeline2Insights – Behavioural interview questions for Data Engineers

https://pipeline2insights.substack.com/p/week-2831-behavioural-interview-questions-for-data-engineers

9) Interviewing.io – FAANG behavioral mock replay

https://interviewing.io/mocks/faang-behavioral-tech-lead-manager

10) YouTube – Behavioral STARs / non-technical data interview prep

https://www.youtube.com/watch?v=tRdLVUKU7Bo

DataGeeks

COPY RIGHTS RESERVED

DataGeeks

COPY RIGHTS RESERVED